“Mini Linux Shell with Background Job Management”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

Chirukumudi Sri Dhruthi **2520030460**
2520030593
Ravi Abhishiktha **2520030185**

Under the Esteemed Guidance of

Dr. K. Anusha
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

We hereby declare that the project entitled “Mini Linux Shell with Background Job Management” is undertaken as part of our Operating Systems Hackathon project at KL (Deemed to be) University. This project focuses on understanding Linux process management, command execution, background jobs, and signal handling through a C-based command-line application.

The implementation details and test results presented in the final report shall reflect the work actually completed by the team.

2520030460
2520030593
Ravi Abhishiktha **2520030185**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled**"Design and Development of an Integrated Tourism Platform for Flights, Trains, Hotels, and Hidden Destinations"** is a bonafide work done and submitted by **Student name – 25200XXXXX ,**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. K. Anusha, Assistant Professor, Department of Computer Science and Engineering, KL (Deemed to be) University, for her guidance and encouragement throughout this project.

We thank our department and university for providing the resources and learning environment required to understand Operating Systems concepts. We also thank our teammates, faculty members, and everyone who supported us in completing this project.

2520030460

2520030593

Ravu Abhishiktha 2520030185

Abstract

The Linux operating system provides a powerful command-line environment for executing programs and managing processes. However, understanding the internal working of a shell requires practical knowledge of operating system concepts. This project, titled “Mini Linux Shell with Background Job Management,” aims to develop a lightweight shell using the C programming language.

The shell accepts user commands, parses the input, and executes external programs using Linux system calls such as `fork()` and `execvp()`. It supports foreground and background process execution, allowing users to run commands and manage multiple tasks. A job table can be used to track background processes, while `waitpid()` and signal handling help monitor process completion.

The project also explores built-in commands, process synchronization, input/output redirection, and pipes where implemented. It is developed for an Ubuntu/Linux environment using the GCC compiler. This project provides practical experience with process creation, execution, termination, signals, and file operations. It helps students understand how a shell interacts with the Linux kernel and demonstrates important concepts of Operating Systems through a simple command-line application.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	**Implementation**		**30–45**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**62–67**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**79–83**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**84–86**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

Chapter 1: Introduction

1.1 Background of the Project

A shell is a command-line interface that allows users to communicate with an operating system. Users enter commands, and the shell interprets those commands and requests the operating system to execute them.

Linux provides several shells, such as Bash. These shells support command execution, process management, pipes, redirection, and other features.

Our project develops a mini shell using C. It focuses on the basic working of a shell and demonstrates how processes are created, executed, monitored, and terminated.

1.2 Problem Statement

Traditional Linux shells provide many features, but beginners may find it difficult to understand how commands are executed internally.

The major problems addressed by this project are:

- Understanding how user commands are parsed and executed.
- Running processes in foreground and background modes.
- Monitoring background processes.
- Handling process termination and signals.
- Understanding how Linux system calls support process management.

Therefore, we propose a simple C-based shell that demonstrates these Operating Systems concepts.

1.3 Objectives of the Project

The main objectives are:

1. To develop a basic command-line shell using C.
2. To execute Linux commands using process-creation and execution system calls.
3. To support foreground and background command execution.
4. To maintain a job table for tracking background processes.
5. To demonstrate process synchronization and signal handling.
6. To explore input/output redirection and pipes where implemented.
7. To gain practical knowledge of Linux process management.

1.4 Scope of the Project

The project focuses on implementing and understanding basic shell functionality in a Linux environment.

Its scope includes:

- Reading and parsing user commands.
- Executing external programs.
- Handling built-in commands such as `cd` and `exit`.
- Running background jobs.
- Monitoring process status.
- Handling supported signals.
- Demonstrating pipes and redirection if included in the implementation.

Advanced features such as command history, tab completion, and complete Bash-compatible scripting may be considered future enhancements.

1.5 Importance of the Application

This project is useful for students learning Operating Systems because it connects theoretical concepts with practical implementation. It explains how the shell communicates with the Linux kernel through system calls and how multiple processes can be managed efficiently.

Chapter 2: OS Concepts and Background Study

2.1 Process Management

A process is a program that is currently executing. Linux creates processes to run applications and commands.

In our project, the shell creates child processes to execute external commands. The parent shell manages these processes and waits for foreground processes to finish.

2.2 Process Creation — `fork()`

The `fork()` system call creates a new child process.

- The parent process is the shell.
- The child process executes the requested external command.
- The return value helps identify whether the code is running in the parent or child.
- If process creation fails, the shell should report an error.

2.3 Program Execution — `execvp()`

The `execvp()` function replaces the current process program with another executable program.

For example, when a user enters `ls`, the child process can use `execvp()` to execute the Linux `ls` command.

If execution fails, the program should display an appropriate error message.

2.4 Process Synchronization — `waitpid()`

The `waitpid()` system call allows a parent process to wait for a child process and collect its termination status.

- For foreground jobs, the shell waits until the child finishes.
- For background jobs, the shell can use the `WNOHANG` option to check whether a child has finished without blocking.
- Collecting terminated child processes helps prevent zombie processes.

2.5 Signal Handling

Signals notify processes about particular events.

Signal	Purpose
`SIGINT`	Commonly generated by Ctrl+C

`SIGTSTP` Commonly generated by Ctrl+Z

`SIGCHLD` Notifies the parent about a child process state change

The `sigaction()` function can configure signal handling. Signal handlers should perform only safe operations, leaving complex job-table updates to the main shell loop where appropriate.

2.6 Foreground and Background Processes

Foreground process: The shell waits for the command to finish before returning to the prompt.

Background process: The command runs while the shell returns to the prompt, allowing the user to enter another command.

Example:

```
sleep 10
```

This runs in the foreground when entered normally.

```
sleep 10 &
```

This requests background execution.

2.7 Inter-Process Communication (IPC)

Inter-Process Communication allows processes to exchange information.

A pipe is one mechanism used to pass the output of one command to another command. For example:

```
ls | wc
```

Here, the output of `ls` is passed to `wc` through a pipe. This feature applies to our shell only if pipe handling has been implemented.

Chapter 3: System Requirements

3.1 Hardware Requirements

- Computer or laptop.
- Minimum 2 GB RAM recommended.
- Keyboard and display.
- Sufficient storage for source code and development tools.

3.2 Software Requirements

Component	Requirement
Operating system	Ubuntu/Linux or compatible WSL environment
Programming language	C
Compiler	GCC
Code editor	Visual Studio Code or another text editor
Terminal	Ubuntu/Linux terminal
Debugging tool	GDB, optional
Version control	Git, optional

3.3 Operating System Environment

The application is designed for a Linux/POSIX environment. Ubuntu provides the system calls, compiler, and terminal utilities needed to develop and test the shell.

Windows users can use WSL with Ubuntu installed.

Chapter 4: Technology and Tools Used

4.1 C Programming Language

C is used to implement the shell because it provides access to low-level operating system interfaces and process-management functions.

4.2 GCC Compiler

GCC converts the C source code into an executable program.

Example:

```
gcc -Wall -Wextra -o minishell minishell.c
```

The filename should be changed to match the actual source file.

4.3 Linux System Calls and Functions

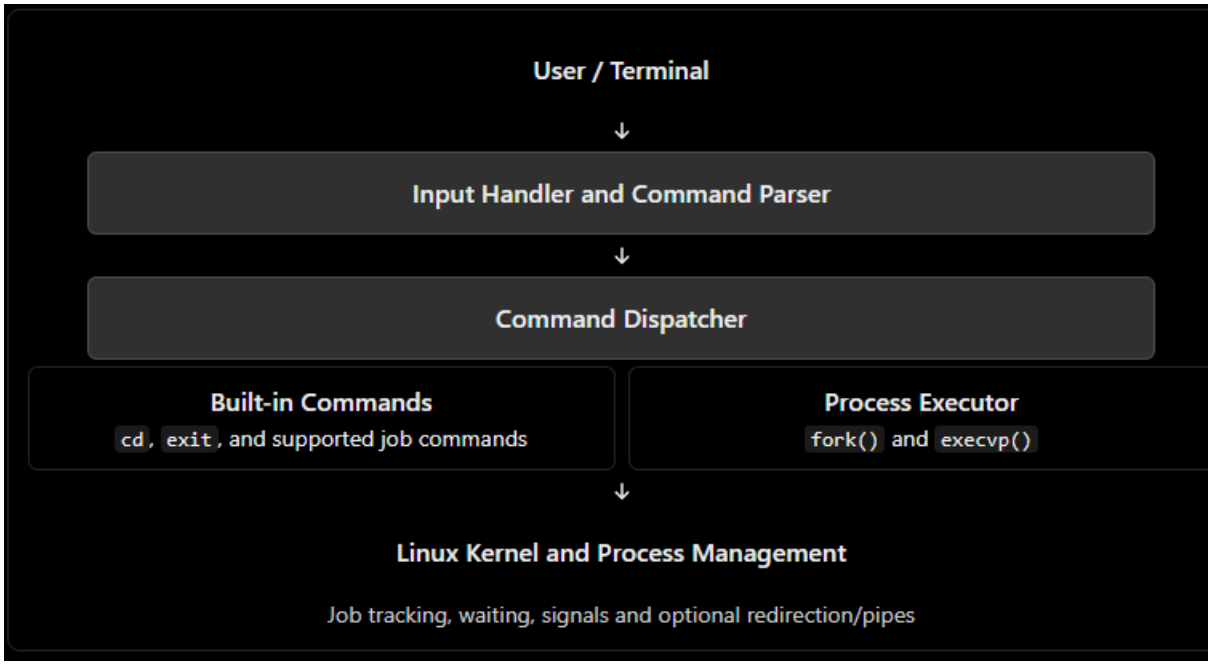
Function	Purpose
`fork()`	Creates a child process
`execvp()`	Executes an external program
`waitpid()`	Waits for or checks child processes
`chdir()`	Changes the current working directory
`getcwd()`	Retrieves the current working directory
`open()`	Opens a file
`dup2()`	Redirects a file descriptor
`pipe()`	Creates a communication channel
`kill()`	Sends a signal to a process
`sigaction()`	Configures signal handling

These functions are relevant to the design; the final report should list only those actually used in the submitted code.

Chapter 5: System Architecture and Design

5.1 Overall System Architecture

The system contains the following logical components:



Chapter 6: Implementation

6.1 Main Shell Loop

The shell repeatedly accepts input and processes commands. The loop continues until the user enters the supported exit command or another defined termination condition occurs.

6.2 Foreground Execution

When the user enters an external command without &, the shell creates a child process and waits for its completion using `waitpid()`.

6.3 Background Execution

When a command ends with &, the shell should execute it in the background and return to the prompt without waiting for normal completion.

6.4 Pseudocode

START SHELL

WHILE shell is running:

Display prompt

Read user command

Parse command and arguments

IF command is empty:

Continue

IF command is a built-in:

Execute built-in command

ELSE:

Create child process using fork()

IF process creation fails:

Display error

ELSE IF current process is child:

Execute program using execvp()

Display error if execution fails

ELSE IF command is foreground:

Wait using waitpid()

ELSE:

Add process to job table

Return to prompt

Check completed background jobs

END WHILE

STOP SHELL

6.5 Error Handling

The shell should handle invalid commands, empty input, process-creation failures, and program-execution failures. It should display understandable error messages and continue operating when appropriate.

Chapter 7: Execution and Demonstration

7.2 Suggested Test Cases

Test command	Expected result
`pwd`	Displays the current directory
`ls`	Lists files and directories
`echo Hello`	Displays the given text
`cd ..`	Changes the shell's directory
`sleep 10 &`	Returns to the prompt while the job runs
`jobs`	Displays tracked jobs, if implemented
`ls > out.txt`	Redirects output to a file, if implemented
`ls | wc`	Connects command output through a pipe, if implemented
Invalid command	Displays an error without crashing

Record Pass or Fail only after actually running each test in your shell.

7.3 Screenshots

Insert screenshots of your own execution showing:

- The shell prompt and basic commands.
- Foreground command execution.
- Background command execution.
- Job listing, if implemented.
- Redirection and pipe output, if implemented.

- Error handling for an invalid command.

Chapter 8: Team Contributions

The following responsibilities are suggested based on your team of three. Update them to reflect each member's actual contribution.

Team member	Suggested contribution
Cherukumudi Sri Dhruthi	Input handling, command parsing and built-in commands
Kachana Mahima	Process execution, foreground/background execution and report preparation
Ravi Abhishiktha	Background job management, signal handling and testing

All team members can contribute to integration, debugging, testing, and final presentation preparation.

Chapter 9: Results, Limitations and Future Enhancements

9.1 Expected Results

The project aims to demonstrate how a basic Linux shell accepts and executes commands, creates processes, handles foreground and background tasks, and monitors process completion.

The actual results must be recorded after testing the submitted implementation.

Number of test cases passed: To be filled after testing.

Features successfully demonstrated: To be filled after testing.

9.2 Limitations

- The mini shell may support fewer commands and features than Bash.
- Advanced command parsing and shell scripting may not be supported.
- Complete foreground/background terminal job control requires additional process-group and terminal management.
- Redirection, pipes, and commands such as `fg` and `bg` work only if implemented correctly.

9.3 Future Enhancements

- Add command history.
- Implement tab completion.
- Improve command parsing and error handling.
- Improve foreground and background job control.
- Add more advanced pipe and redirection support.
- Add automated testing and build automation using a Makefile.

Chapter 10: Conclusion

The Mini Linux Shell with Background Job Management project provides practical experience with important Operating Systems concepts. By designing a shell in C, the team studies command parsing, process creation, program execution, process synchronization, signal handling, and background job management.

The project connects theoretical knowledge with practical Linux programming and improves understanding of how applications interact with the operating system. Future enhancements can make the shell more reliable and provide additional command-line features.

References

1. Michael Kerrisk, The Linux Programming Interface, No Starch Press.
2. GNU Compiler Collection (GCC) documentation: GCC Documentation.
3. Linux manual pages: Linux man-pages.
4. The Open Group, POSIX specifications: POSIX Documentation.

.